

Welcome to **Theory of Computation**

CS 420

UMass Boston Computer Science

Instructors: Stephen Chang

Fall 2026

Today's Theme:

What's this course about?

Welcome to **Theory of Computation**

CS 420

UMass Boston Computer Science

Instructors: Stephen Chang

Fall 2026

What's this?



Interlude: Lecture Logistics

- *I expect:* lecture to be interactive
 - Participation is a part of your grade
 - Also, it's the best way to learn!
- *I may:* call on students randomly
 - It's ok to be wrong in class! – will not affect your grade
 - Also, it's the best way to learn!
- *Please:* tell me your name before speaking
 - Sorry in advance if I get it wrong
 - Also, it's the best way for me to learn!

Welcome to **Theory of Computation**

CS 420

UMass Boston Computer Science

Instructors: Stephen Chang

Fall 2026



How would you
define this?

Computation Is ... (via examples)

- $1 + 1 = ??$
- $= 2$

... some basic definitions and assumptions (“**axioms**”),
e.g., define “Numbers” to be: 0, 1, 2, 3, ...

- $11 + 11 = ??$
- $= 22$

... and rules that use the definitions and axioms (“**algorithm**”),
e.g., grade school arithmetic

- $9999999999 + 9999999999 = ??$
- $= 19999999998$

Computation rules can be executed by
hand, or by **machine / automaton**



- $1 + 1 = ??$
- $= 10$

(binary)

There are many possible definitions
(i.e., **models**) of **computation**

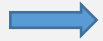


(hint)

Computation Is ... Programs!

```
def bigger(x):  
    if x > 0:  
        return x + 1  
    else:  
        return x - 1
```

```
print( bigger(10) )
```



11

Result???

You already use
**models of
computation!**
Every time you
reason about code!

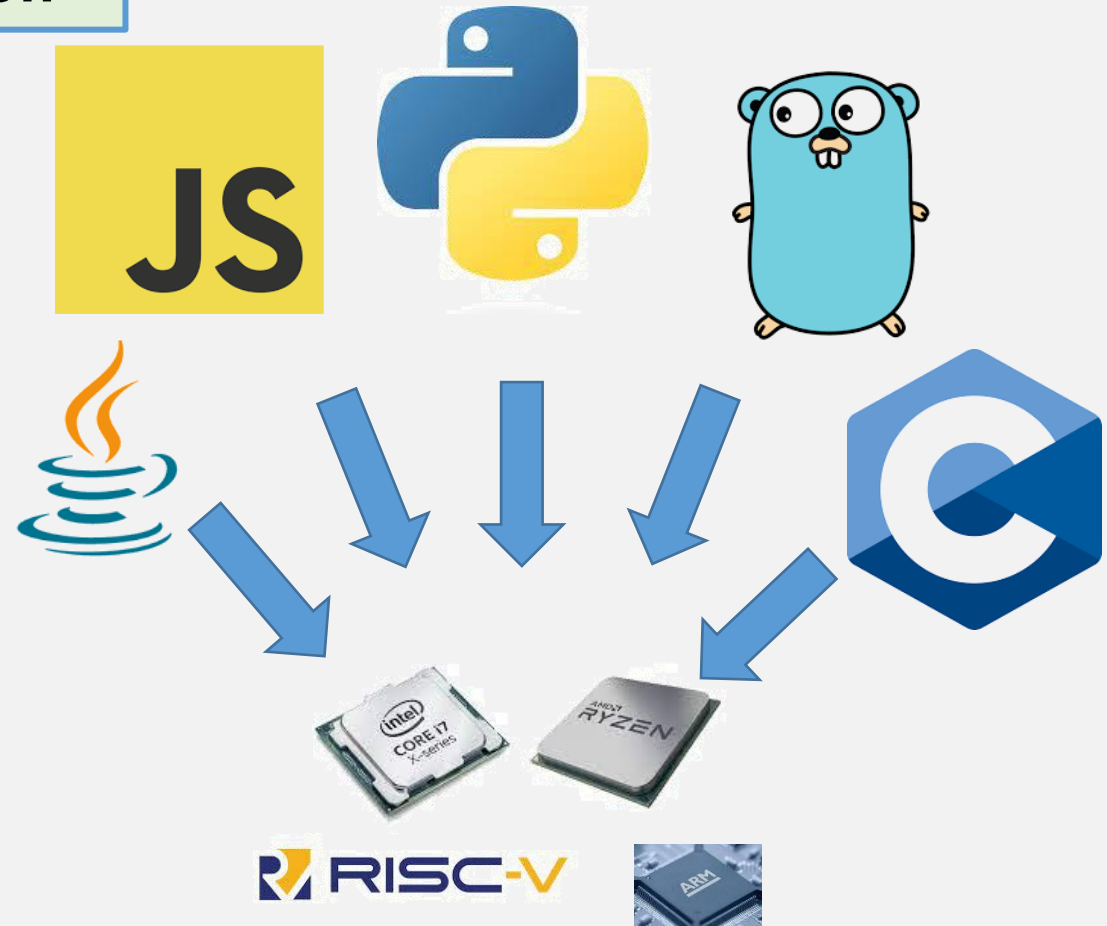
Every programming language
is a **model of computation**

different???

If they are different:
how can we know?

Or same???

If they are the same:
is there a common
model for all?

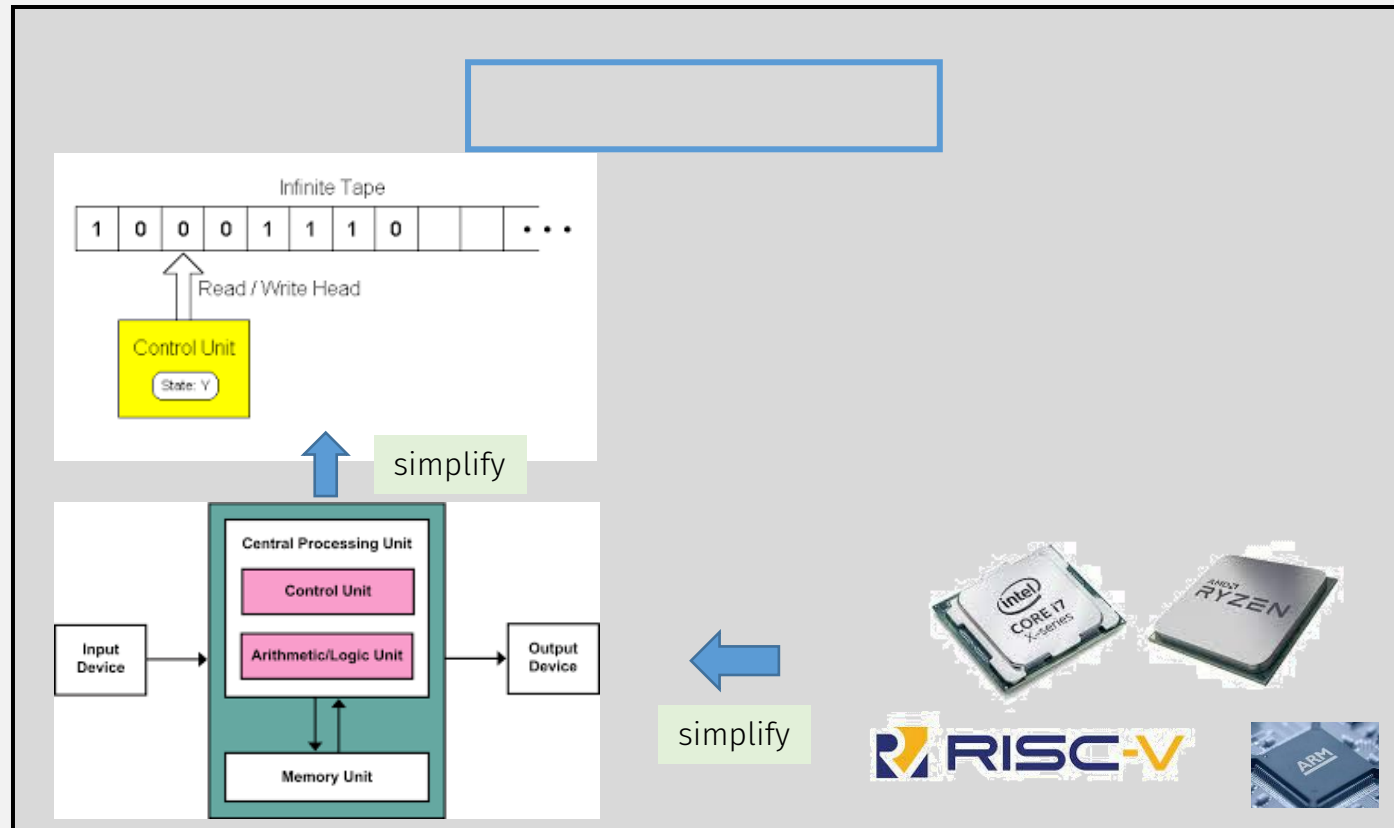


This semester, we will ...

1. Define and study **models of computation**

- models will be *as simple as possible* (to make them easier to study)

Models of Computation (spoiler alert!)



This semester, we will ...

1. Define and study **models of computation**

- models will be *as simple as possible* (to make them easier to study)

2. Compare and contrast models of computation

- which “programs” are *included* by a model
- which “programs” are *excluded* by a model
- *overlap* between models?

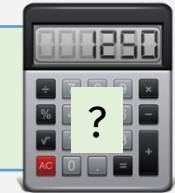
Models of Computation

Turing Machines

Q₁: Are there computational models ... other than Turing Machines?

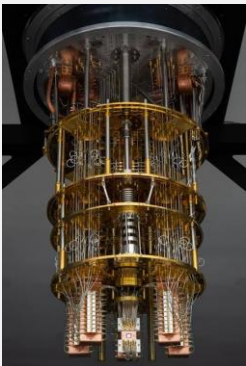
Q₂: Are there computational models ... *"weaker"* than Turing Machines?

Q₃: Are there computational models ... *"more powerful"* than Turing Machines?



Q₄: What does *"weaker"* or *"more powerful"* even mean?!

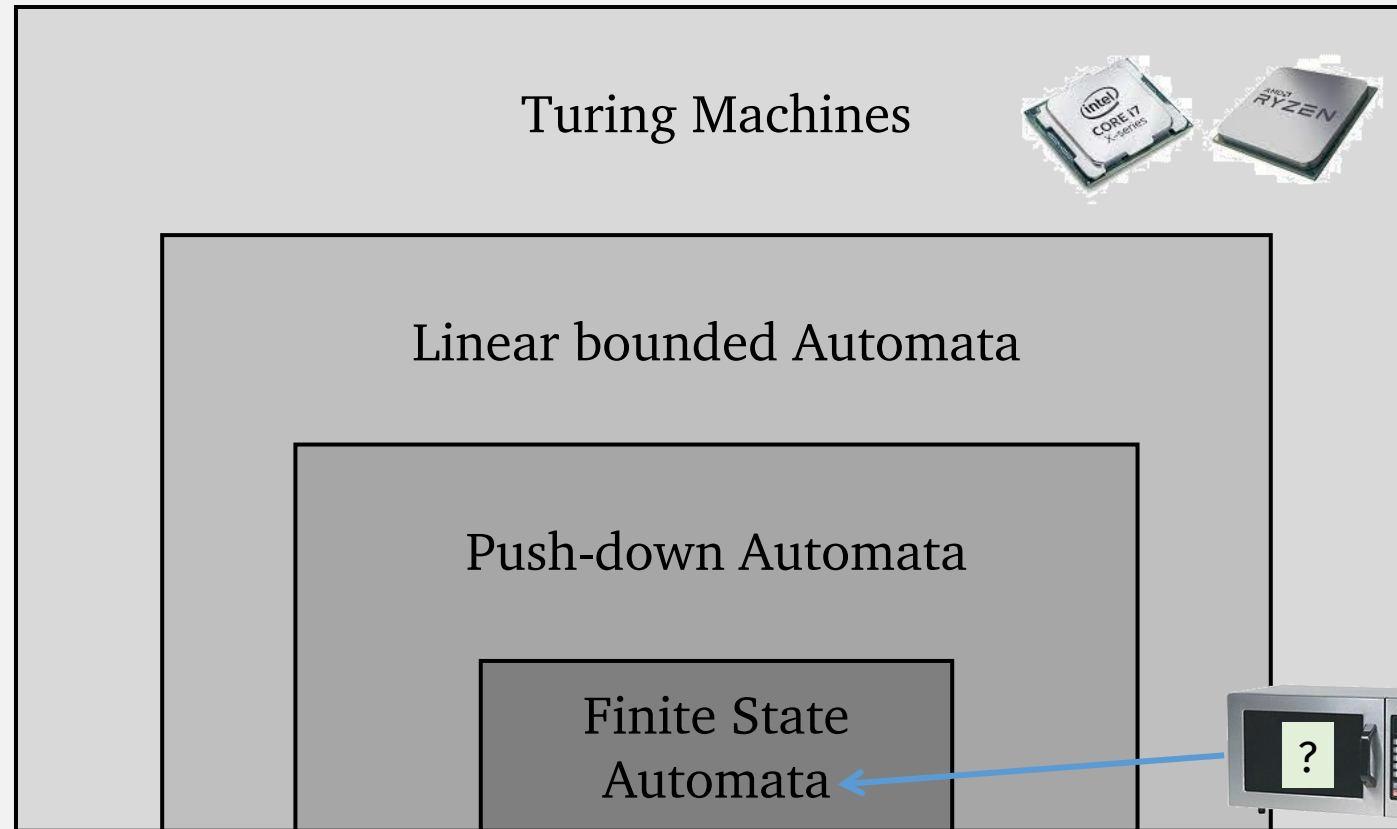
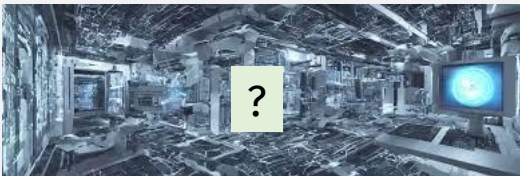
A: Yes, yes, yes, and ... stay tuned!



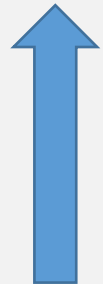
Models of Computation Hierarchy

... and get to here ...

... and also look at
what's out here???

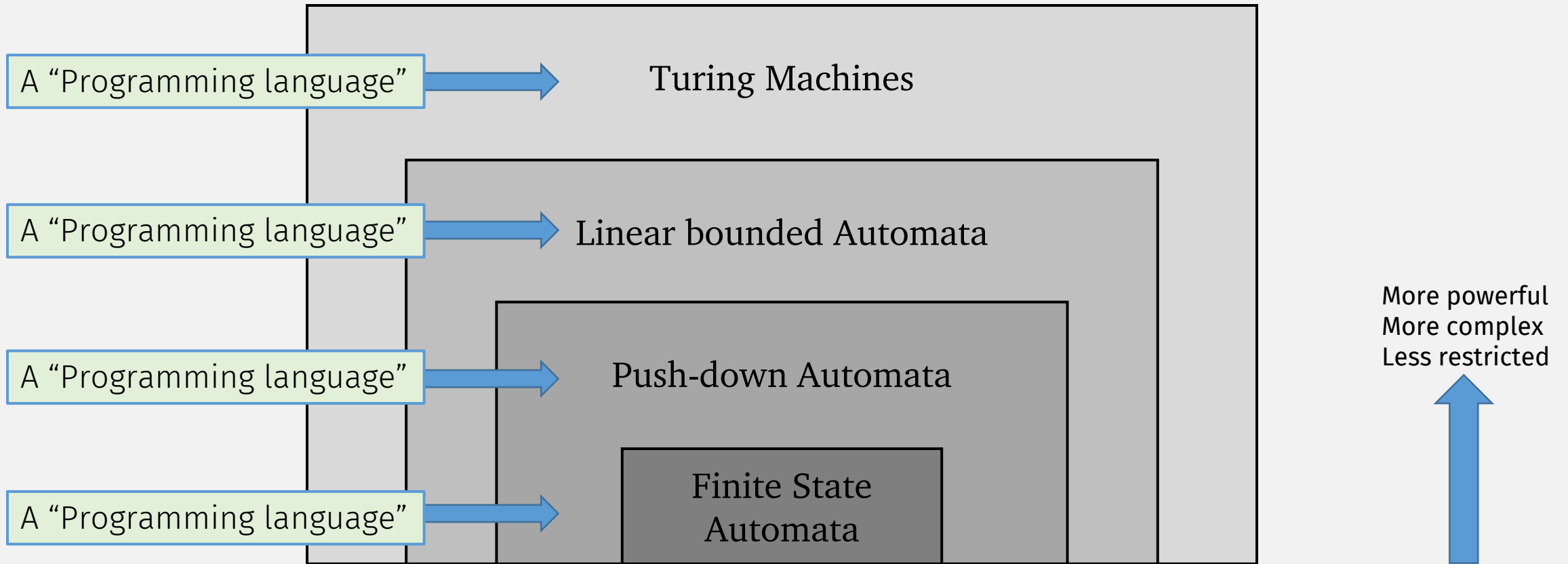


More powerful
More complex
Less restricted



We'll start here ...

But remember ... Computation = Programs!



Helpful analogy for this course:

- a **set** of machines / computational model (a rectangle above) ~ a **Programming Language!**
- a **single** machine (a thing in a rectangle above) ~ a **Program!**

What's this? ☒

Welcome to Theory of Computation

CS 420

UMass Boston Computer Science

Instructors: Stephen Chang

Fall 2026

What's this?



Welcome to **Theory** of Computation

CS 420

UMass Boston Computer Science

Instructors: Stephen Chang

Fall 2026

“Theory” = math
(This is a math course!)

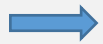
(But programming is math too!)

Programming Is (What) Math?

Math(ematical) logic!

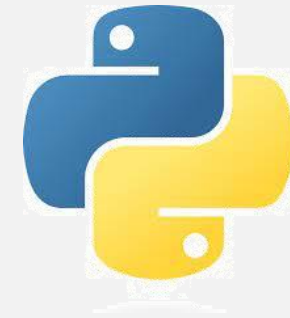
```
def bigger(x):  
    if x > 0:  
        return x + 1  
    else:  
        return x - 1  
  
print( bigger(10) )
```

Result???



11

How did you figure out the answer?



(But programming is math too!)

Programming = Mathematical logic!

- “logic is the foundation of all computer programming”

- <https://www.technokids.com/blog/programming/its-easy-to-improve-logical-thinking-with-programming/>

- “logic is the fundamental key to becoming a good developer”

- <https://www.geeksforgeeks.org/i-cant-use-logic-in-programming-what-should-i-do/>

- “Analytical skill and logical reasoning are prerequisites of programming because coding is effectively logical problem solving at its core”

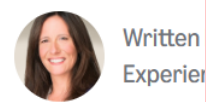
- <https://levelup.gitconnected.com/the-secret-weapon-of-great-software-engineers-22d57f427937>

(Studying logic, i.e., this class!, will make you a better programmer!)

“But I don’t need to know how to program anymore!”

--- someone who doesn’t want to work as a programmer???

**I'm an Amazon SVP who hadn't coded in 25 years.
Then I wrote 100,000 lines of code with AI.**



Written
Experien

AWS suffered 'at least two outages' caused by AI tools, and now I'm convinced we're living inside a 'Silicon Valley' writes 100% of implications

By Jason England | Published February 20, 2026

FINANCIAL TIMES
Amazon holds engineering meeting following AI-related outages

Ecommerce giant says there has been a 'trend of incidents' linked to 'Gen-AI assisted changes'

Rafe Rosner-Uddin in San Francisco

Published MAR 10 2026 | Updated MAR 10 2026, 12:50

FORTUNE

Amazon puts humans further back in the loop as its retail website crashes from 'inaccurate advice' that an AI agent took from an old wiki

By Eva Roytburg

Fellow, News

March 12, 2026, 2:14 PM ET



We're All Doomed? ... again???

Citrini Research

THE 2028 GLOBAL INTELLIGENCE CRISIS

A Thought Exercise



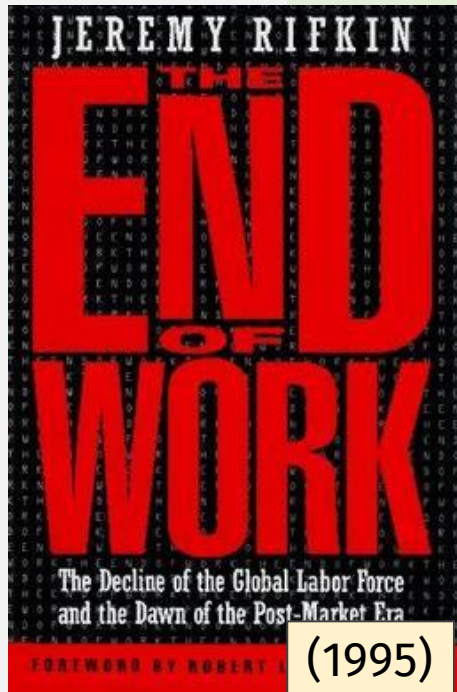
CITRINI AND A

FEB 22, 2026

AI got better and cheaper. Companies laid off workers, then used the savings to buy more AI capability, which let them lay off more workers. Displaced workers spent less.

Companies that sell things to consumers sold fewer of them, weakened, and invested more in AI to protect margins. AI got better and cheaper.

A feedback loop with no natural brake.



(1995)



(2018)

newsletter.pessimistsarchive.org

WEDNESDAY, MARCH 28

(1928)

UNEMPLOYMENT IS BLAMED ON NEW MACHINES

Devices Displace Workers.

DOES MACHINE DISPLACE MEN IN THE LONG RUN?

(1940s)

New Studies Cited as Old Argument Is Renewed Over Significance of 'Technological Unemployment'

Impact of Automation on the Well-off

(1960s)

Editorial

THE GREAT DILEMMA OF U.S. LABOUR

Automation Might End Most Unskilled Jobs In 10 Years

From A STAFF CORRESPONDENT in New York

IN America today, when management and labour meet to plan their joint future, the time-honoured causes for haggling — strikes and shut-outs and increased wages — are likely to be settled amicably and in a hurry.

The union may be moderate in its wage demands and the company more willing to meet them. The complexities of automation, which are



There's hope ...

(Impossible! And you should know why by the end of semester!)

Why Copilot writes bad code [link](#)

The reason is because of how language models work. They show how, on average, most people write. **They don't have any sense of what's correct or what's good.** Most code on GitHub is (by software standards) pretty old, and (by c

=

Thinking



count to 10 starting from 11

11, 12, 13, 14, 15, 16, 17, 18, 19, 20



should I walk or drive to car wash. It's only 100 feet away

literally about 100 feet away, walking is probably easier

(ChatGPT Getting Worse Over Time?)

Lingjiao Chen[†], Matei Zaharia[‡], James Zou[†]

[†]Stanford University [‡]UC Berkeley

Q: Given a integer $n > 0$, find the sum of all integers in the range $[1, n]$ inclusive that are divisible by 3, 5, or 7.

GPT-3.5

22.0%

2.0%

March 2023

June 2023

ation

worse

Existing Code



In order to
use AI to program,
you first need to
know how to program!

wors
ger



Karri Saarinen ✓
@karrisaarinen

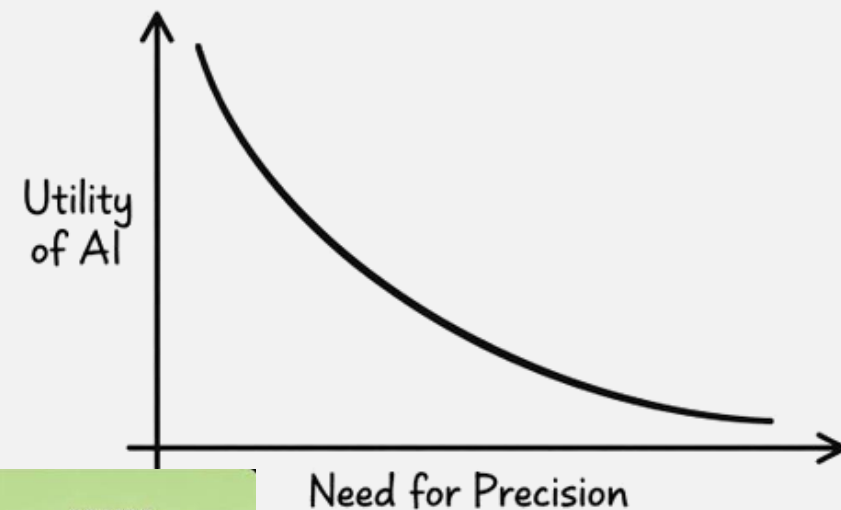


A common dynamic I observe with AI: it feels most impressive when you don't know much about the subject, don't care or don't have a clear idea of what the you want.

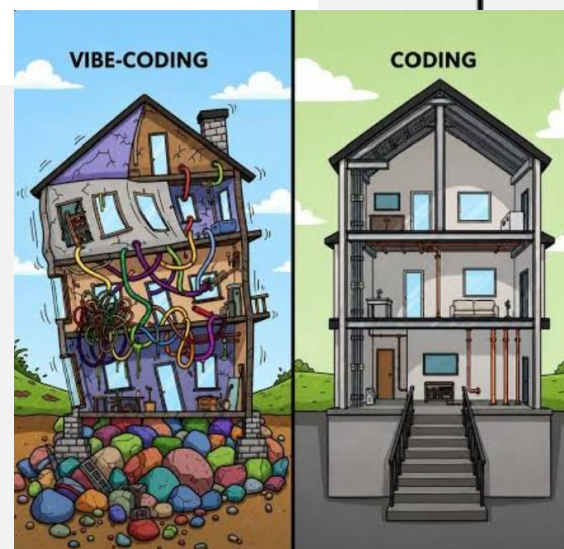
This applies across design, code, legal, and more. If I don't know code very well, every piece of code it writes feels very impressive.

Once you know what something should feel or look like, it becomes almost impossible to guide AI there. And you definitely can't one-shot it.

11:45 AM · Apr 25, 2026 · 565.2K Views



In order to
use AI to program,
you first need to
know how to program!



Who to trust? (You decide)



AGI has arrived. Congratulations @OpenAI team.



FORTUNE

Princeton's 'AI Snake Oil' author says the real fear isn't thinking machines—it's that AI exposes who already knows how to think



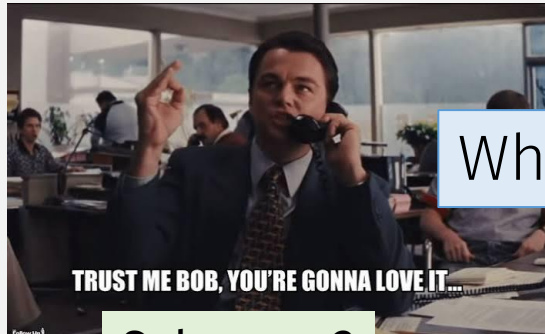
By Nick Lichtenberg
Business Editor



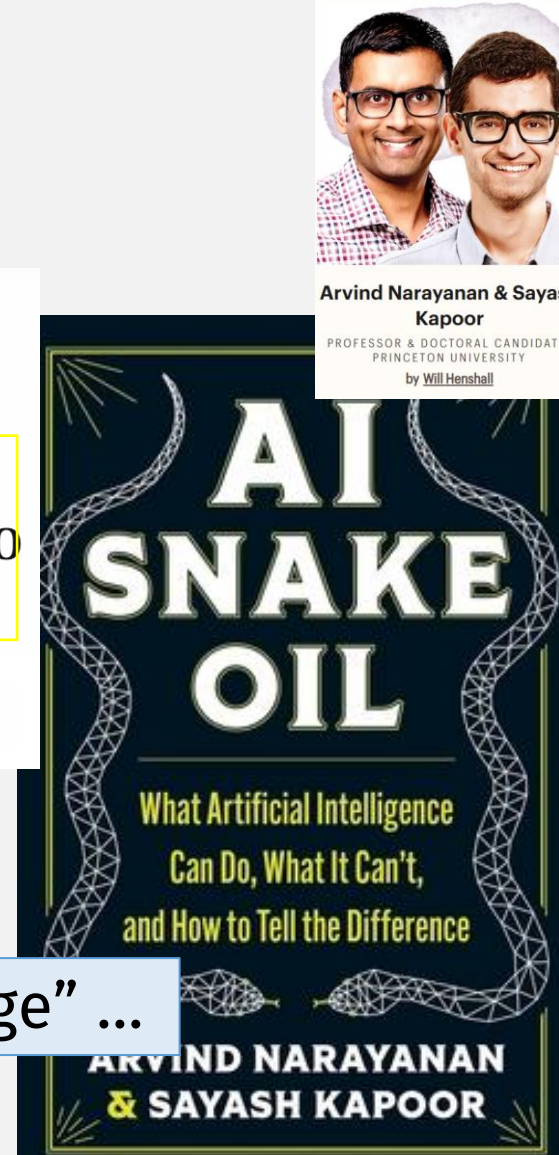
August 19, 2026, 3:00 AM ET

AI makes foundational knowledge more important than ever

Where can students go to get "foundational knowledge" ...



Salesman?



Vs (maybe out-of-touch) academics?

Welcome to **Theory** of Computation

CS 420

UMass Boston Computer Science

Instructors: Stephen Chang

Fall 2026

“Theory” = math

(This is a math course!)

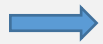
(and programming is math!)

Programming Is (What) Math?

Math(ematical) logic!

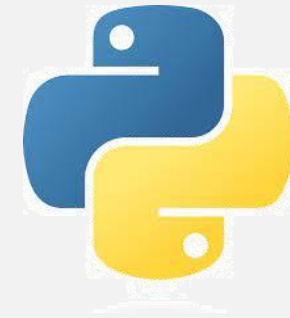
```
def bigger(x):  
    if x > 0:  
        return x + 1  
    else:  
        return x - 1  
print( bigger(10) )
```

???



11

How did you figure out the answer?



(and programming is math!)

Programming = Mathematical logic!

Programming Concepts

- Functions
- Variables
- If-then
- Recursion
- Strings
- **Sets** (and other data structures)
- **Programming** in general???

Math(ematical Logic) Concepts

- Functions
- Variables
- If-then (implication)
- Recursion
- Strings
- **Sets** (and other groupings of data)
- **Proving** theorems!!!

(Studying logic, i.e., this class!, will make you a better programmer!)

Proving = Programming

Proofs are Programs: 19th Century Logic and 21st Century Computing

Philip Wadler
Avaya Labs

June 2000, updated November 2000

As the 19th century drew to a close, logicians formalized an ideal notion of proof. They were driven by nothing other than an abiding interest in truth, and their proofs were as ethereal as the mind of God. Yet within decades these mathematical abstractions were realized by the hand of man, in the digital stored-program computer. How it came to be recognized that proofs and programs are the same thing is a story that spans a century, a chase with as many twists and turns as a thriller. At the end of the story is a new principle for designing programming languages that will guide computers into the 21st century.

For my money, Gentzen's natural deduction and Church's lambda calculus are on a par with Einstein's

This semester, we will ...

1. Define and study **models of computation**

- models will be *as simple as possible* (to make them easier to study)

2. Compare and contrast models of computation

- which “programs” are *included* by a model
- which “programs” are *excluded* by a model
- *overlap* between models?

3. Prove things about the models

By learning to prove ...

... you will **become a better programmer**

Writing proofs (on paper) = **programming** (without an IDE)

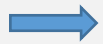
(We will only do this with very short programs / proofs)

but ...

this means that **answers that are not precise are wrong!**

Reasoning About Code is ~~Math~~ Proof

```
def no_div0(x):  
    if (x > 0) | (x < 0) | (x == 0):  
        return x + 1  
    else:  
        return 1 / 0  
  
print( no_div0(10) ) ???
```



11

Can this function ever throw ZeroDivisionError?

No!

How did you figure out the answer?

You used the Python model of computation
to predict the program's behavior

You did a proof!

Clarification note:

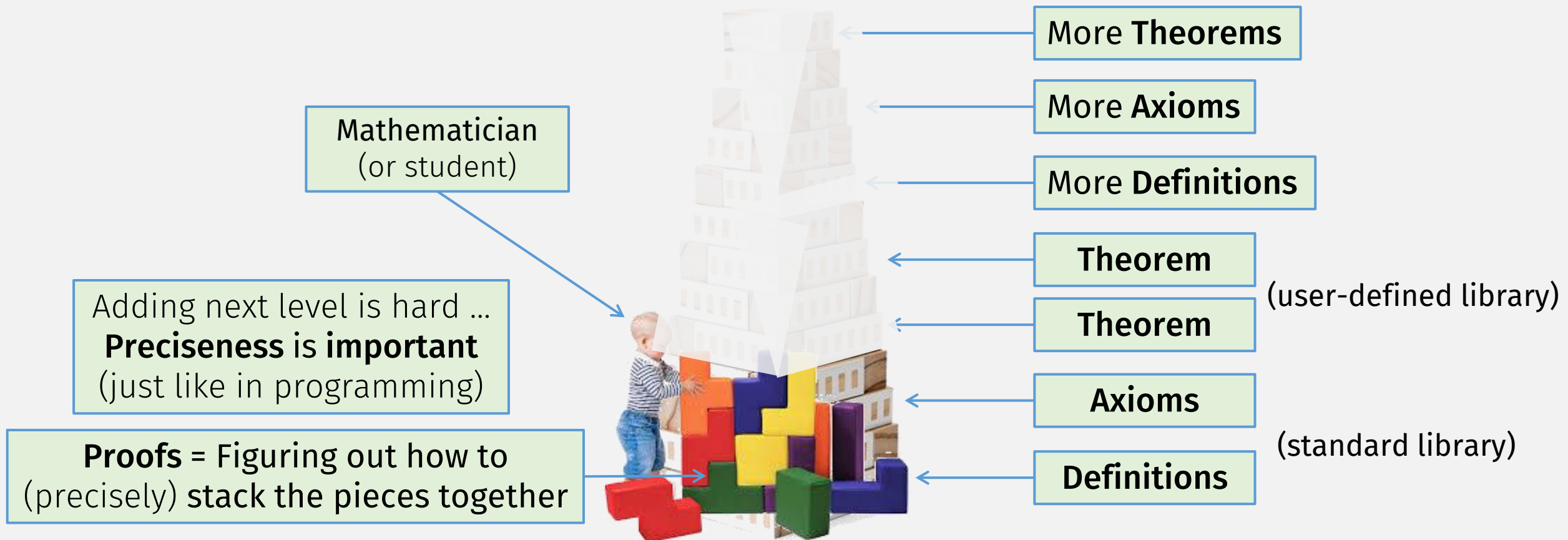
In this course, we will **prove** things about
“programs” / computation ...

... which itself is **equivalent** to writing
“programs” ...

... ie, programs that analyze programs ...



How Mathematics (Proofs) Work



The “Modus Ponens” Inference Rule

(Precisely Fitting Blocks Together)

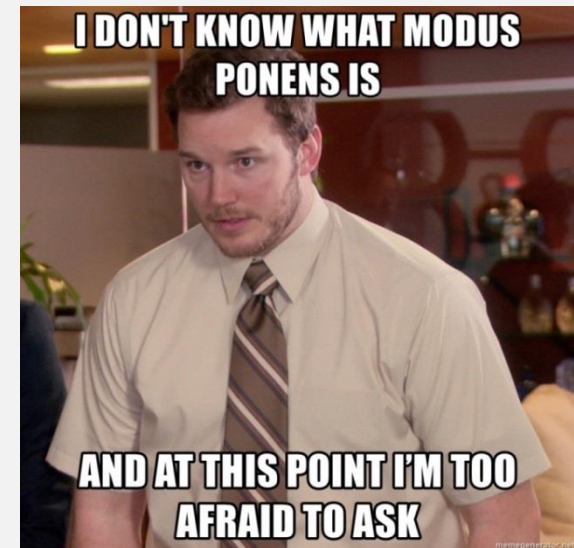
Premises (if we can show these statements are true)

- If P then Q ← More specifically, we say that we **APPLY** (a proof of) this
- P is **TRUE** ← TO (a proof of) this

Conclusion (then we can say that this is also true)

- Q must also be **TRUE** ← to get a (a proof of) this

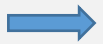
It's exactly like a **function**!
Functions are **APPLIED TO** an **argument** to get a **result**!



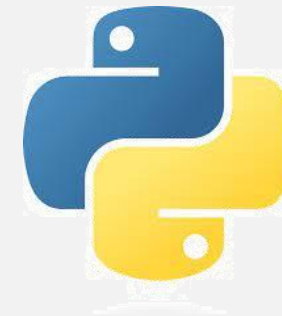
You already do “Proof” when Programming

```
def no_div0(x):  
    if (x > 0) | (x < 0) | (x == 0):  
        return x + 1  
    else:  
        return 1 / 0  
  
print( no_div0(10) )
```

???



11



Can this function ever throw ZeroDivisionError?

No!

How did you figure out the answer?

You used the Python model of computation
to predict the program's behavior

(Let's write it out formally)

You did a proof!

Deductive Proof Example

Prove: `no_div0` never throws `ZeroDivisionError`

```
def no_div0(x): "test expr"  
    if (x > 0) | (x < 0) | (x == 0):  
        return x + 1 "first branch"  
    else:  
        return 1 / 0 "second branch"
```

Proof "construction" :

Prior steps are already-proved, can be used to prove later steps!

Statements / Justifications Table

a proof is made of smaller proofs (each line)

Statements

- Proof₁
1. IF running "test expr" is True,
THEN "first branch" runs
 2. IF running "test expr" is False,
THEN "second branch" runs
- Proof₃
3. running "test expr" is (always) True
- 4. "first branch" (always) runs

Justifications

1. Rules of Python
(“built-in” facts)
2. Rules of Python
3. Definition of “numbers”
4. APPLY Proof₁ TO Proof₃

Modus Ponens

If we can prove these:

- If P then Q

P

Then we've proved:

- Q ←

It's exactly like a function!

Functions are APPLIED TO an argument to get a result!

7. `no_div0` never throws `ZeroDivisionError`

(ends with the Statement we want to prove)

Deductive Proof Example

Prove: `no_div0` never throws `ZeroDivisionError`

```
def no_div0(x):  
    if (x > 0) | (x < 0) | (x == 0):  
        return x + 1  
    else:  
        return 1 / 0
```

“second branch”

Proof “construction” :

Statements

1. IF running “test expr” is True,
THEN “first branch” runs
2. IF running “test expr” is False,
THEN “second branch” runs
3. running “test expr” is (always) True
- Proof₄ 4. “first branch” (always) runs
5. “second branch” *never* runs
6. `no_div0` never runs `1 / 0`
- ➔ 7. `no_div0` never throws `ZeroDivisionError`

Justifications

1. Rules of Python
2. Rules of Python
3. Definition of “numbers”
4. APPLY Proof₁ TO Proof₃
5. Using Proof₄ and Rules of Python???
6. Using Proof₅ and ... ?
7. Using Proof₆ and Rules of Python???

But what's the
inference rule?

What else can we prove about programs?

```
function check(n)
{ // check if the number n is a prime
  var factor; // if the checked number is not a prime, this is its first factor
  var c;
  factor = 0;
  // try to divide the checked number by all numbers till its square root
  for (c=2; (c <= Math.sqrt(n)); c++)
  {
    if (n%c == 0) // is n divisible by c ?
    { factor = c; break }
  }
  return (factor);
} // end of check function

function communicate()
{ // communicate with the user
  var i; // i is the checked number
  var factor; // if the checked number is not a prime, this is its first factor
  i = document.primetest.number.value; // get the checked number
  // is it a valid input?
  if ((i != N(i)) || (i <= 0) || (Math.floor(i) != i))
  { alert("The checked object should be a whole positive number"); }
  else
  {
    factor = check(i);
    if (factor == 0)
    { alert(i + " is a prime number"); }
    else
    { alert(i + " is not a prime number. i =" + factor + "X" + i/factor); }
  }
} // end of communicate function
```



Proof = prediction about program result
... without running the program

Can we make predictions about computation?



It's tricky: Trying to predict
computation requires computation!

Can we make predictions about computation?

- The **Halting Lemma** says:



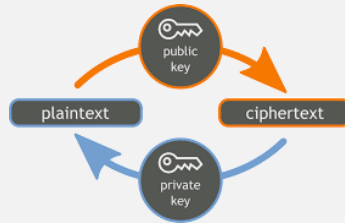
- And **Rice's Theorem** says:

- “all non-trivial, semantic properties of programs are undecidable”

Knowing What Computers Can't Do is Still Useful!

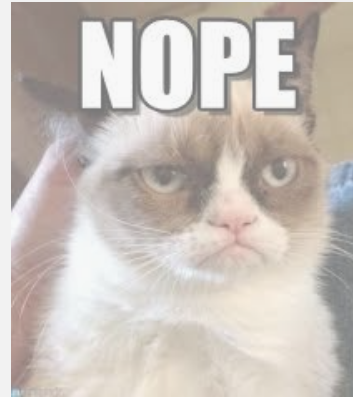
In Cryptography:

- Perfect secrecy is impossible in practice
- But with slightly imperfect secrecy (i.e., a computationally bounded adversary) we get:



Can we make predictions about computation?

- The **Halting Lemma** says:



- And **Rice's Theorem** says:

- “all non-trivial, semantic properties of programs are undecidable”

Actually:

- it depends on the computation model!



Predicting What Some Programs Will Do ...

microsoft.com/en-us/research/project/slam/

SLAM is a project for checking that software satisfies critical behavioral properties of the interfaces it uses and to aid software engineers in designing interfaces and software that ensure reliable and correct functioning. Static Driver Verifier is a tool in the Windows Driver Development Kit that uses the SLAM verification engine.

"Things like even software verification, this has been the Holy Grail of computer science for many decades but now in some very key areas, for example, driver verification we're building tools that can do actual proof about the software and how it works in order to guarantee the reliability." **Bill Gates**, April 18, 2002. [Keynote address at WinHec 2002](#)

SLAM

Predicting things about programs ... is the Holy grail of CS!

Static Driver Verifier Research Platform README

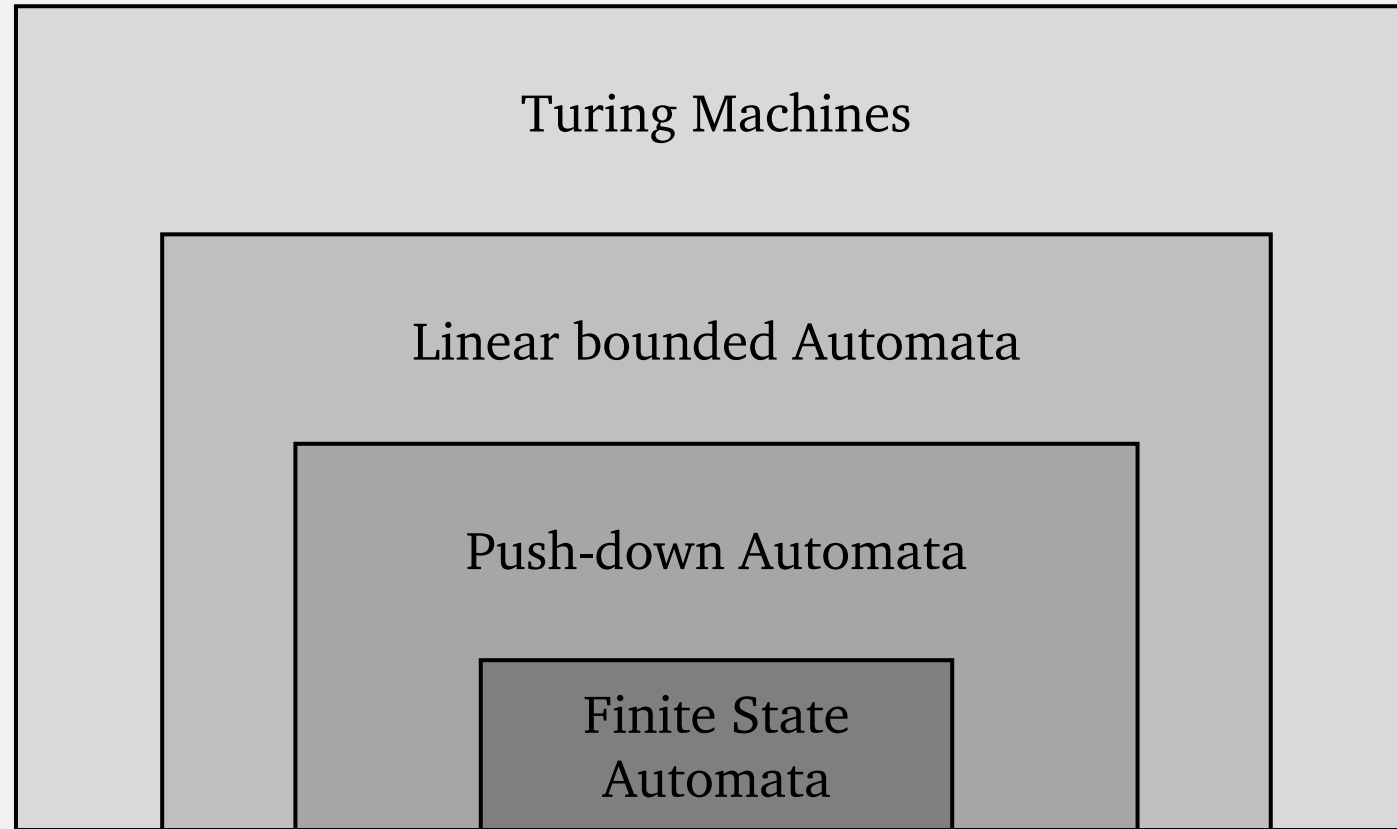
Overview of Static Driver Verifier Research Platform

Static Driver Verifier (SDV) is a compile-time static verification tool, included in the Windows Driver Kit (WDK). The SDV Research Platform (SDVRP) is an extension to SDV that allows you to adapt SDV to:

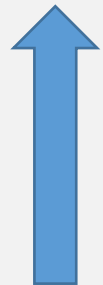
- Support additional frameworks (or APIs) and write custom SLIC rules for this framework.
- Experiment with the model checking step.



Proofs About Computational Models ... in this class

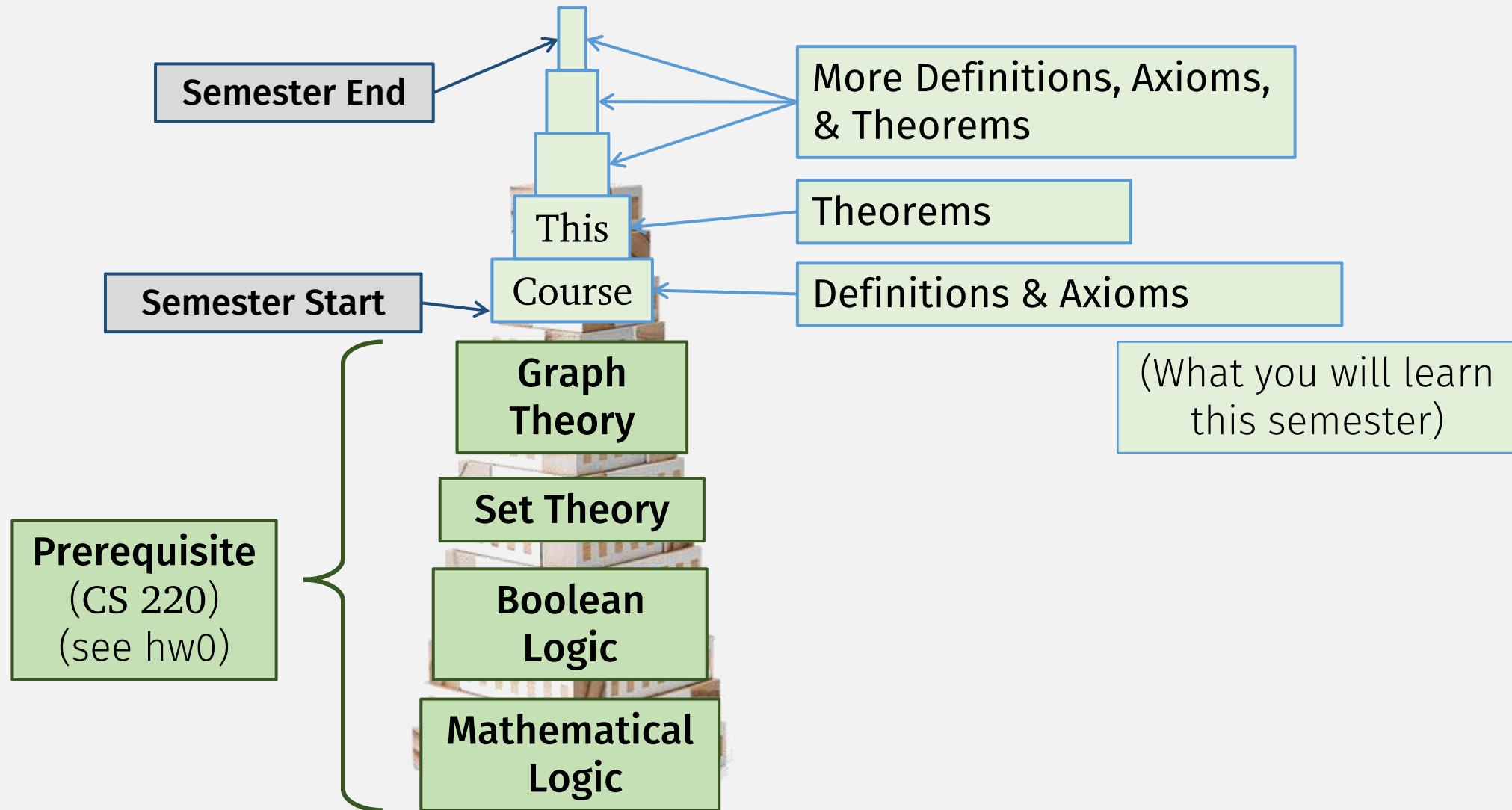


More powerful
More complex
Less restricted



In this class, we will **prove** things about simple computational models (not Python ...)

How This Course Works



A Word of Advice

Important:
Do not fall behind
in this course



To prove a (new) theorem ...

... need to know all axioms,
definitions, and (previous)
theorems below it

Another Word of Advice

HW 1, Problem 1

Prove that $ABC = XYZ$



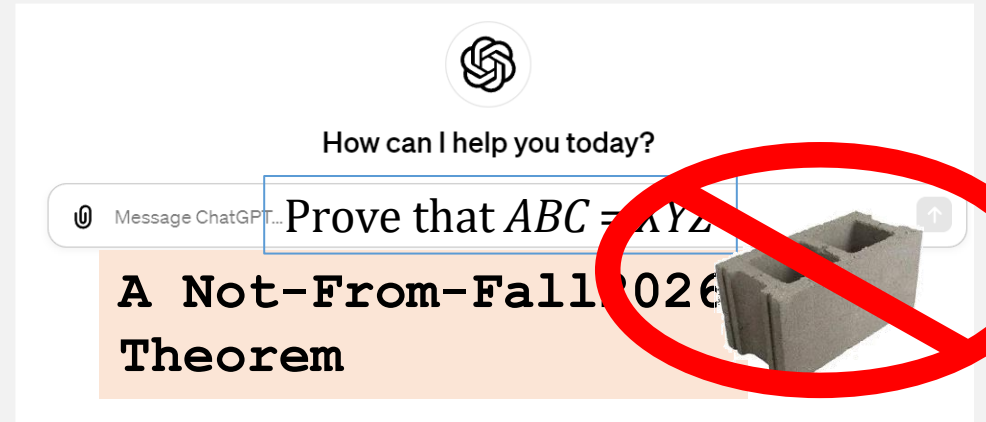
“Blocks” from outside the course won’t work in the proof

Remember:
Preciseness in proofs (just like in programming) **is critical**
(Proofs must connect facts from this course exactly)

HW problems are *graded* on precise steps in the proof, not on the final theorem itself!

... can be used to **prove** (new) **theorems** in this course

Only **axioms, definitions,** and **theorems** from this course...



Textbooks

- Sipser. *Intro to Theory of Computation*, 3rd ed.
- Hopcroft, Motwani, Ullman. *Intro to Automata Theory, Languages, and Computation*, 3rd ed.
- **Slides** (posted) and **lecture** will try to be **self-contained**,
- BUT, **students who read the book earn higher grades**

All course info available on **web site**:

- cs.umb.edu/~stchang/cs420/f26

How to Do Well in this Course

- Learn the “building blocks”
 - I.e., axioms, definitions, and theorems
- To solve a problem (prove a new theorem) ...
... think about how to (precisely) combine existing “blocks”
- HW problems graded on steps to the answer (not final theorem)
- Don't Fall Behind!
 - Start HW Early (HW 0 due Tuesday 9/14 12pm EST noon)
- Participate and Engage
 - Lecture
 - Office Hours
 - Message Boards (piazza)

Grading

- **HW: 80%**
 - Weekly: In / Out Monday
 - Approx. 12 assignments
 - Lowest grade dropped
- **Participation: 20%**
 - Lecture participation, in-class work, office hours, piazza
- **No exams**
- **A range: 90-100**
- **B range: 80-90**
- **C range: 70-80**
- **D range: 60-70**
- **F: < 60**

All course info available on web site:
- cs.umb.edu/~stchang/cs420/f26

Late HW

- Is bad ... try not to do it please
 - Grades get delayed
 - Can't discuss solutions
 - You fall behind!
- Late Policy: **3 late days** to use during the semester
 - Note: this policy is independent from what the “late due date” says in GradeScope

HW Collaboration Policy

Allowed

- Discussing HW with classmates (but must cite)
- Using other resources to learn, e.g., youtube, other textbooks, ...
- Writing up answers on your own, from scratch, in your own words

Not Allowed

- Submitting someone else's answer
- Submitting someone else's answer with:
 - variables changed,
 - thesaurus words,
 - or sentences rearranged ...
- Using sites like Chegg, CourseHero, Bartleby, Study, ChatGPT, etc.
- Using theorems or definitions not from this course

Honesty Policy

- 1st offense: zero on problem
- 2nd offense: zero on hw, reported to school
- 3rd offense+: F for course

Regret policy

- If you self-report an honesty violation, you'll only receive a zero on the problem and we move on.

All Up to Date Course Info

Survey, Schedule, Office Hours, HWs, ...

See course website(s):

All course info available on web site:
cs.umb.edu/~stchang/cs420/f26

hw0 (pre-req quiz)
(see gradescope)